

EXP.NO: 6	IPC using Shared Memory
DATE:	

AIM:

To implement a C program that facilitates communication between two independent processes (a Sender and a Receiver) using System V Shared Memory. The objective is to allow the Sender to write a string into a shared memory segment and the Receiver to read and display that same string.

ALGORITHM:

1. Identify the Segment

Both processes define a unique **Key** (e.g., 1234). This acts as a common ID so they can find the same memory location.

2. Create / Access (`shmget`)

The **Sender** creates a 1024-byte memory segment using `shmget` with the `IPC_CREAT` flag.

The **Receiver** uses the same key to locate the existing segment.

3. Attach to Address Space (`shmat`)

Both processes "map" the shared memory into their own memory space. This returns a pointer (`str`), allowing them to interact with the shared memory as if it were a local variable.

4. Data Exchange

Sender: Takes user input via `scanf` and writes it to the memory pointed to by `str`.

Receiver: Reads the string directly from the same pointer `str` and prints it.

5. Detach (`shmdt`)

The **Receiver** detaches from the memory segment once the reading is complete to ensure clean termination.

CODE:

```
// sender

#include <stdio.h>    // for printf, scanf
#include <sys/ipc.h>  // for key_t
```

```

#include <sys/shm.h> // for shared memory functions
#include <string.h> // for string operations

int main() {
    key_t key = 1234; // Unique key to identify shared memory
    int shmid; // Variable to store shared memory ID

    // Create shared memory segment of size 1024 bytes
    // 0666 = read/write permission for all users
    // IPC_CREAT = create memory if it doesn't exist
    shmid = shmget(key, 1024, 0666 | IPC_CREAT); // Create

    // Attach shared memory to this process
    // Returns pointer to the shared memory
    char *str = (char*) shmat(shmid, (void*)0, 0); // Attach

    printf("Enter message: ");

    // Read a full line (including spaces) and store directly in shared memory
    scanf("%[^\n]", str);

    // Print the data written into shared memory
    printf("Data written to shared memory: %s\n", str);

    return 0; // End program
}

```

//receiver

```

#include <stdio.h> // for printf
#include <sys/ipc.h> // for key_t
#include <sys/shm.h> // for shared memory functions

int main() {
    key_t key = 1234; // Same key used by sender
    int shmid; // Variable to store shared memory ID

    // Access existing shared memory
    // No IPC_CREAT → it must already exist
    shmid = shmget(key, 1024, 0666); // Get

    // Attach shared memory to this process
    char *str = (char*) shmat(shmid, (void*)0, 0); // Attach

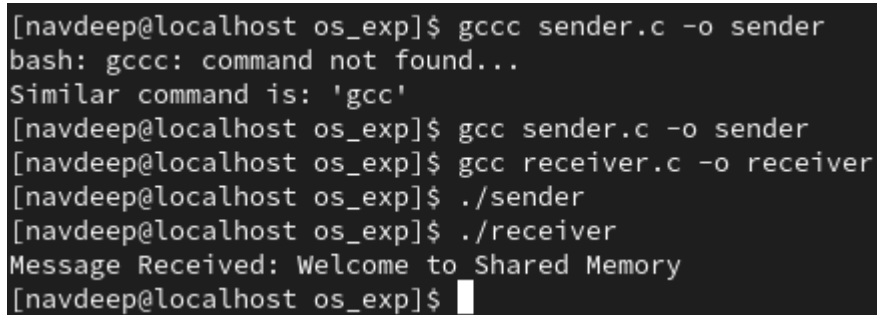
```

```
// Read and print data from shared memory
printf("Data read from shared memory: %s\n", str);

// Detach shared memory after use
shmdt(str);

return 0; // End program
}
```

OUTPUT:



```
[navdeep@localhost os_exp]$ gcc sender.c -o sender
bash: gcc: command not found...
Similar command is: 'gcc'
[navdeep@localhost os_exp]$ gcc sender.c -o sender
[navdeep@localhost os_exp]$ gcc receiver.c -o receiver
[navdeep@localhost os_exp]$ ./sender
[navdeep@localhost os_exp]$ ./receiver
Message Received: Welcome to Shared Memory
[navdeep@localhost os_exp]$
```

RESULT:

The program successfully simulates process synchronization. The producer is restricted from adding items when the buffer is full ($e=0$), and the consumer is restricted from removing items when the buffer is empty ($f=0$), maintaining data integrity within the shared resource.